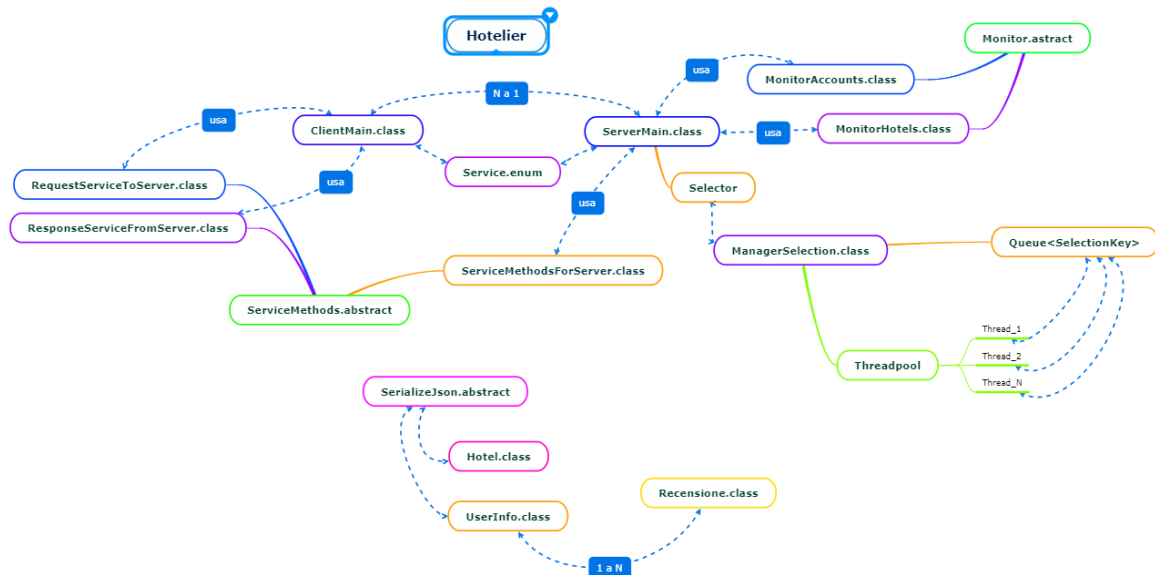


# Hotelier



## Scelte di architettura:

-Per gestire le connessioni con gli utenti, ho deciso di usare il multiplexing dei canali mediante NIO. Nel **ServerMain.class** abbiamo il thread principale (master) che crea il selector, una coda sincronizzata delle selectkeys, un monitor per gestire gli accounts, un monitor per gestire gli hotels e poi un threadpool, a numero fisso di thread. Il thread principale si occupa di aspettare le richieste dai client, ritrovandole sottoforma di selectkeys quando viene chiamato **selectnow()**, in seguito vengono inserite nella coda delle selectkeys. Ogni thread (slave) aspetta la selectkey dalla coda (ordine FiFo) delle selectkeys e quando ne prende una, si occupa di soddisfare la richiesta del client. i threads slave possono accedere alle risorse condivise attraverso i monitor che gli vengono passati nel costruttore.

Il ServerMain.class se quando chiama il metodo selectnow(), viene restituito 0, allora ,rispettando comunque l'intervallo di tempo passato come parametro all'esecuzione del programma, salva in memoria permanente gli accounts degli utenti registrati e gli hotels aggiornati con le nuove recensioni, inoltre calcola di nuovo il ranking per vedere se un nuovo hotel è diventato primo nella classifica e nel caso lo comunica attraverso il canale udp.

-Il ClientMain.class una volta effettuata la connessione tcp/ip attraverso il SocketChannel per mandare al server le richieste e ricevere le risposte, manda in esecuzione un thread che si occupa invece di stabilire una connessione udp per accedere al gruppo di multicast, fatta dal server, e continuare ad aspettare notifiche (è una architettura di tipo publish-subscribe, dove il server dopo ogni intervallo di tempo, se c'è stato un cambiamento, manda al gruppo la notifica sull'hotel con il punteggio del ranking più alto).

---

### **Scelta algoritmo ranking locale:**

viene usata una classe temporanea Rank per gestire il punteggio di uno hotel su una classifica. La formula per calcolare il punteggio tiene in considerazione la qualità e la quantità di recensioni fatte all'hotel.

ValoreScaglione: n° recensioni:

- da 1 a 99 -> 1 punto
- da 100 a 499 -> 2 punti
- da 499 a 1499 -> 3 punti
- dopo 1500 -> 4 punti.

MoltiplicatoreCostante = 2

Punteggio = (punteggio sintetico dell'hotel \* MoltiplicatoreCostante) +  
ValoreScaglione(n°recensioni) + n°recensioni/n°recensioni+1.

Casi test (simulazione):

|       |  |  |        |
|-------|--|--|--------|
| input |  |  | output |
|-------|--|--|--------|

| n recensioni | punteggio sintetico | risultato intermedio | valore scaglione |             |
|--------------|---------------------|----------------------|------------------|-------------|
| 98           | 5                   | 10,90740741          | 1                | 11,90740741 |
| 300          | 4                   | 8,967741935          | 2                | 10,96774194 |
| 600          | 4                   | 8,983606557          | 3                | 11,98360656 |
| 1200         | 2                   | 4,991735537          | 3                | 7,991735537 |
| 1700         | 1                   | 2,994152047          | 4                | 6,994152047 |
| 4000         | 1                   | 2,997506234          | 4                | 6,997506234 |
| 4000         | 2                   | 4,997506234          | 4                | 8,997506234 |
| 2            | 5                   | 10,16666667          | 1                | 11,16666667 |
| 1            | 5                   | 10,09090909          | 1                | 11,09090909 |

---

## Istruzioni per compilare:

### -librerie esterne:

-gson-2.8.2.jar

-parametri: (notare che gli argomenti sono facoltativi perché se non inseriti vengono usati dei parametri preimpostati.)

CLIENT: argC1 -> IP, argC2 -> PORT, argC3 -> PORT\_UDP, argC4 -> IP\_GROUP  
 SERVER: argS1 -> PORT, argS2 -> PORT\_UDP, argS3 -> IP\_GROUP, argS4 -> PORT\_GROUP,  
 argS5 -> INTERVALLO\_RANK\_STORE, argS6 -> INTERVALLO\_SLEEP\_SELECT

### -per compilare:

```
javac -cp ./gson-2.8.2.jar ServerMain.java
```

```
javac -cp ./gson-2.8.2.jar ClientMain.java
```

### -per eseguire:

```
java -cp ./gson-2.8.2.jar ServerMain [argS1 argS2 argS3 argS4 argS5 argS6]
```

```
java -cp ./gson-2.8.2.jar ClientMain [argC1 argC2 argC3 argC4]
```

### -per ottenere jar:

```
jar cvf ServerMain.jar *.class lib/gson-2.8.2.jar
```

```
jar cvf ClientMain.jar *.class lib/gson-2.8.2.jar
```

-per eseguire il jar:

```
java -cp ServerMain.jar:lib/gson-2.8.2.jar ServerMain [argS1 argS2 argS3  
argS4 argS5 argS6]
```

```
java -cp ClientMain.jar:lib/gson-2.8.2.jar ClientMain [argC1 argC2 argC3  
argC4]
```